Cairo University
Faculty of Engineering
Electronics & Communications Dept.

NN ASSIGNMENT 2

Artificial Neural Networks

Technical Report

Prepared by:

Abdallah Essam Mohamed Mahmoud	ID: ...
Zeyad Antar Othman Abu-Zaid	ID: 9220331
Abdelrahman Hatem Nasr Youssef	ID: 9220461

Submitted to:

Prof. Dr. Mohsen Rashwan

Dr. Mohamed Abdelghani

April 2026

Contents

1 Problem 1: Training Feed-Forward Neural Networks (MLP)

1.1 Introduction

This part explores the classification of the **ReducedMNIST** handwritten digit dataset using **Multilayer Perceptrons (MLPs)**. The objective is to evaluate how *network depth* influences performance when combined with different input feature representations.

A Multilayer Perceptron is a fully connected feed-forward neural network. Every neuron in one layer connects to every neuron in the next layer. It is the simplest form of "deep" learning:

- **Input layer:** receives the feature vector (e.g. 225 DCT coefficients).
- **Hidden layers:** apply a learned linear transformation followed by a non-linear activation function (ReLU in our case). More hidden layers = deeper network = ability to learn more complex patterns.
- **Output layer:** 10 neurons (one per digit 0–9), producing class probabilities via the Softmax function.

We trained MLPs with **1, 3, and 4 hidden layers** using features derived from:

1. **Discrete Cosine Transform (DCT)** — 225 features (top-left 15×15 block).
2. **Principal Component Analysis (PCA)** — 262 components retaining 95% of variance.
3. **Autoencoder (AE)** — a neural network trained to compress images into 64 features.

The dataset is a reduced version of the standard MNIST with 10,000 training images (1,000 per digit) and 2,000 test images (200 per digit), each 28×28 pixels normalised to $[0, 1]$.

1.2 Results

System: Intel Core i7-9850H @ 2.60 GHz, 32 GB RAM, Windows 11, Python 3.12, PyTorch 2.11 (CPU only).

Hyper-parameters: Epochs = 10, Batch size = 64, Learning rate = 10^{-3}, Optimizer = Adam, Activation = ReLU, Loss = CrossEntropyLoss. Autoencoder pre-training: 20 epochs, bottleneck = 64, MSE loss.

Hidden Layers	DCT		PCA		AutoEncoder	
	Acc (%)	Time (ms)	Acc (%)	Time (ms)	Acc (%)	Time (ms)
1 layer [128]	95.8	3758.3	93.5	4470.1	96.4	3944.1
3 layers [256-128-64]	96.4	6096.5	93.8	5808.2	**97.2**	6135.4
4 layers [256-128-64-32]	95.5	7154.5	93.3	7322.9	96.7	6417.3

Table 1. Test accuracy and processing time for each feature–depth combination.

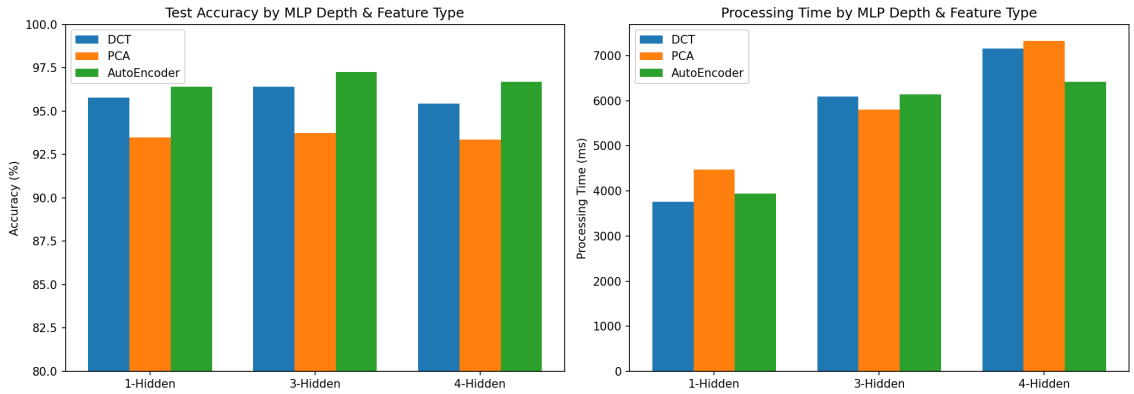

Figure 1. Comparison of test accuracy (left) and processing time (right) across all feature types and MLP depths.

1.3 Analysis and Discussion

1.3.1 Effect of Network Depth and Overfitting

Increasing depth from 1 to 3 hidden layers gave a modest accuracy boost for DCT (95.8% → 96.4%) and AutoEncoder (96.4% → 97.2%), but going to 4 hidden layers *decreased* accuracy in every feature type — DCT dropped to 95.5%, PCA to 93.3%, and AutoEncoder to 96.7%.

This accuracy drop at 4 layers is a clear sign of **overfitting**.

1.3.2 Effect of Feature Type

The **Autoencoder features achieved the highest accuracy** (97.2%) despite being the smallest representation (only 64 dimensions).

Ranking: AutoEncoder (97.2%) > DCT (96.4%) > PCA (93.8%)

Why AutoEncoder outperforms:

- The autoencoder learns a **non-linear** mapping, allowing it to capture complex relationships between pixels that linear PCA cannot.

- Its 64-D features are extremely compact yet highly discriminative — the bottleneck forces it to retain only the most class-relevant information.
- The autoencoder is trained *on the same data*, so its features are tailored to this specific dataset.

Why DCT outperforms PCA:

- DCT captures frequency information which is well-suited for digit recognition (global shape patterns).
- PCA with 262 components retains some noisy dimensions, which can slightly confuse the classifier.

1.3.3 Processing Time

Processing time scales with both **network depth** and **input dimensionality**:

- PCA (262-D input) consistently took the longest due to the larger first weight matrix.
- AutoEncoder (64-D input) was the fastest despite achieving the best accuracy — demonstrating that good feature engineering reduces both compute cost and error rate.
- Going from 1 to 4 hidden layers roughly doubles the processing time (e.g. DCT: 3758 → 7155 ms), while accuracy does not improve proportionally.

2 Problem 4: Speech Digit Recognition using Autoencoders

2.1 Introduction

This problem explores **speech digit recognition** (digits 0–10) using audio recordings. The goal is twofold: (1) build a **baseline** classifier from hand-crafted audio features, and (2) show that an **autoencoder** can learn a superior latent representation that significantly outperforms the baseline.

Key concepts:

- **Utterance:** one complete audio recording of a person saying a digit.
- **Frame:** a short 15 ms slice of the waveform (with 10 ms hop). Speech is approximately stationary over such short durations.
- **MFCC:** Mel-Frequency Cepstral Coefficients — 30 coefficients per frame that mimic how the human ear perceives pitch and timbre.
- **Delta & Delta-Delta:** first- and second-order time derivatives of MFCCs that capture velocity and acceleration of spectral change.

The dataset contains **1,200 training** and **300 testing** utterances (files named `{SpeakerID}_{digit}.wav`). All utterances have 81 frames after MFCC extraction, so the flattened input is $30 \times 81 = 2{,}430$ dimensions.

2.2 Baseline: Summary Features + SVM

The baseline extracts frame-level MFCC, delta, and delta-delta coefficients from each utterance and summarises them into a fixed-length feature vector by computing global statistics (mean and standard deviation) across all frames. This collapses the variable-length temporal signal into a compact representation that captures the average spectral characteristics of each digit while discarding frame ordering.

2.3 Autoencoder Approach

2.3.1 Architecture

The supervised autoencoder processes the full flattened utterance ($\mathbb{R}^{2430}$) and consists of:

- **Encoder:** $2430 \to 1024 \to 512 \to 256$
- **Decoder:** $256 \to 512 \to 1024 \to 2430$
- **Classifier head:** Linear($256 \to 10$) branching from the latent layer

2.3.2 Training

The autoencoder is trained with a **joint loss**:

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \lambda \cdot \mathcal{L}_{\text{cls}} \quad \text{where} \quad \mathcal{L}_{\text{recon}} = \text{MSE}(\mathbf{x}, \hat{\mathbf{x}}), \quad \mathcal{L}_{\text{cls}} = \text{CrossEntropy}(\mathbf{y}, \hat{\mathbf{y}})$$

with $\lambda = 2.0$ to emphasise classification.

Data augmentation: 3 noisy copies of each training sample (Gaussian noise, $\sigma = 0.05$) expand the training set from 1,200 to **4,800 samples**.

Training runs for **500 epochs** with Adam (lr = 10^{-3}), batch size 32.

2.3.3 Classification Strategies

Three strategies are evaluated on the test set:

A. Classifier head: direct output of the AE's built-in linear classifier.

B. SVM on latent vectors: train an SVM on the 256-D encoder output.

C. SVM on hybrid features: concatenate the scaled 256-D latent vector with the scaled 180-D summary features ($\mathbb{R}^{436}$) and train an SVM.

2.4 Results

System: same as Problem 1.

Method	Accuracy (%)	Train Time (s)	Test Time (ms)
Baseline (Summary + SVM)	70.00	10.5	60.2
AE Classifier Head	86.00	2225.2	15.3
AE Latent + SVM	89.33	2244.1	85.6
AE Hybrid + SVM (best)	**90.67**	2255.6	92.9

Table 2. Problem 4: Accuracy comparison of baseline vs. autoencoder-based strategies. AE training time includes 500 epochs of AE training plus SVM GridSearchCV.

2.5 Analysis and Discussion

2.5.1 Why the Autoencoder Outperforms the Baseline

The baseline achieves **70.00%** by averaging MFCCs across time, which *discards all temporal information* — two utterances with identical average spectra but different time evolution become

indistinguishable.

The autoencoder processes the **full frame sequence** (2,430-D input) and learns a 256-D latent code that captures both **spectral content** and **temporal dynamics**, yielding **90.67%** accuracy — a **+20.67 percentage point** improvement.

2.5.2 Hybrid Features

The best strategy combines the autoencoder latent vector with the summary features (each independently standardised).

2.5.3 Data Augmentation

Gaussian noise augmentation (3×) expanded the training set from 1,200 to 4,800 samples, which:

- Reduces overfitting in the 500-epoch training regime.
- Teaches the AE to be robust to noise, improving generalisation.